

PROYECTO FINAL

Rendimiento con Algoritmos de Ordenamiento y Búsqueda en Datos Reales

Gestion de productos de una tienda en linea

Lenguaje: Java 17

Fecha de entrega: 15 de agosto de 2026

Estudiante: Adrian H.

Asignatura: Algoritmos y Estructuras de Datos

Resumen ejecutivo

Este proyecto implementa un modulo experimental para administrar un catalogo de 50 productos de una tienda en linea. Cada producto contiene id, nombre, precio, categoria, stock y calificacion promedio. Sobre el mismo conjunto de datos se compararon Merge Sort, Quick Sort y Heap Sort usando dos criterios, ademas de una busqueda binaria por id y una busqueda lineal por subcadena de nombre.

La medicion se realizo con System.nanoTime(), calentamiento previo de la maquina virtual de Java y multiples repeticiones sobre copias identicas del catalogo. En esta ejecucion, Quick Sort fue el mas rapido al ordenar por precio y Heap Sort al ordenar por calificacion. Para veinte consultas, la busqueda binaria por id fue aproximadamente 115 veces mas rapida que la busqueda lineal por nombre. Los valores son propios del entorno de prueba y pueden variar entre equipos.

1. Introduccion

Una tienda en linea necesita presentar productos ordenados y localizar informacion con rapidez. El objetivo es aplicar algoritmos clasicos a objetos reales, medir su comportamiento y relacionar los resultados empiricos con la complejidad teorica. A diferencia de ordenar enteros aislados, aqui cada comparacion accede a un atributo de un objeto Producto mediante un Comparator.

1.1 Objetivos

- Modelar y generar 50 productos aleatorios, coherentes y reproducibles.
- Implementar Merge Sort, Quick Sort y Heap Sort sin utilizar el ordenamiento incorporado de Java.
- Ordenar por precio ascendente y calificacion promedio descendente.
- Implementar busqueda binaria por id y busqueda lineal por subcadena de nombre.
- Medir, comparar, interpretar y documentar los resultados.

2. Modelo y generacion de datos

La clase Producto es inmutable: sus seis atributos se asignan en el constructor y se consultan mediante metodos get. Los ids se generan entre 1001 y 1050 y luego se mezclan, por lo que siguen siendo unicos aunque el catalogo inicial no este ordenado. Se utilizo la semilla fija 20260815 para repetir exactamente los mismos datos.

Atributo	Tipo	Descripcion
id	int	Identificador unico
nombre	String	Producto, marca y variante
precio	double	Valor entre B/.5.00 y B/.500.00
categoria	String	Electronica, Ropa, Libros u Hogar
stock	int	Unidades entre 0 y 200
calificacionPromedio	double	Puntuacion entre 1.0 y 5.0

2.1 Muestra del catalogo generado

ID	Nombre	Precio	Categoria	Stock	Calif.
1003	Organizador Modular Nova 1	B/.225.48	Hogar	73	3.3
1026	Ratón Inalámbrico Apex 2	B/.388.83	Electrónica	80	1.3
1019	Auriculares Bluetooth Apex 3	B/.468.33	Electrónica	170	1.7
1004	Cuentos Infantiles Lumen 4	B/.113.73	Libros	178	1.7
1027	Auriculares Bluetooth Terra 5	B/.428.47	Electrónica	164	1.1
1037	Vestido Casual Apex 6	B/.490.29	Ropa	133	2.9
1008	Zapatillas Urbanas Terra 7	B/.73.73	Ropa	47	3.0
1039	Camisa de Algodón Apex 8	B/.85.51	Ropa	15	2.4

El archivo resultados/productos.csv incluido en el proyecto contiene los 50 registros completos.

3. Algoritmos de ordenamiento

Merge Sort divide la lista, ordena cada mitad y las combina. Garantiza $O(n \log n)$ y usa memoria auxiliar $O(n)$. Quick Sort particiona alrededor de un pivote; su promedio es $O(n \log n)$, aunque puede degradarse a $O(n^2)$. Heap Sort construye un monticulo maximo y extrae elementos; garantiza $O(n \log n)$ y trabaja con memoria adicional $O(1)$.

Las implementaciones son genericas y reciben un Comparator. El criterio de precio compara `getPrecio()` de menor a mayor. El criterio de calificacion compara `getCalificacionPromedio()` en orden inverso.

3.1 Metodologia de medicion

Cada algoritmo se calento durante 500 ejecuciones y se midio 5,000 veces. En cada repeticion se creo una copia de la lista original, pero el cronometro solo abarco el ordenamiento. Al final se comprobo automaticamente el orden. El promedio reduce el ruido que produciria una unica medicion de apenas 50 objetos.

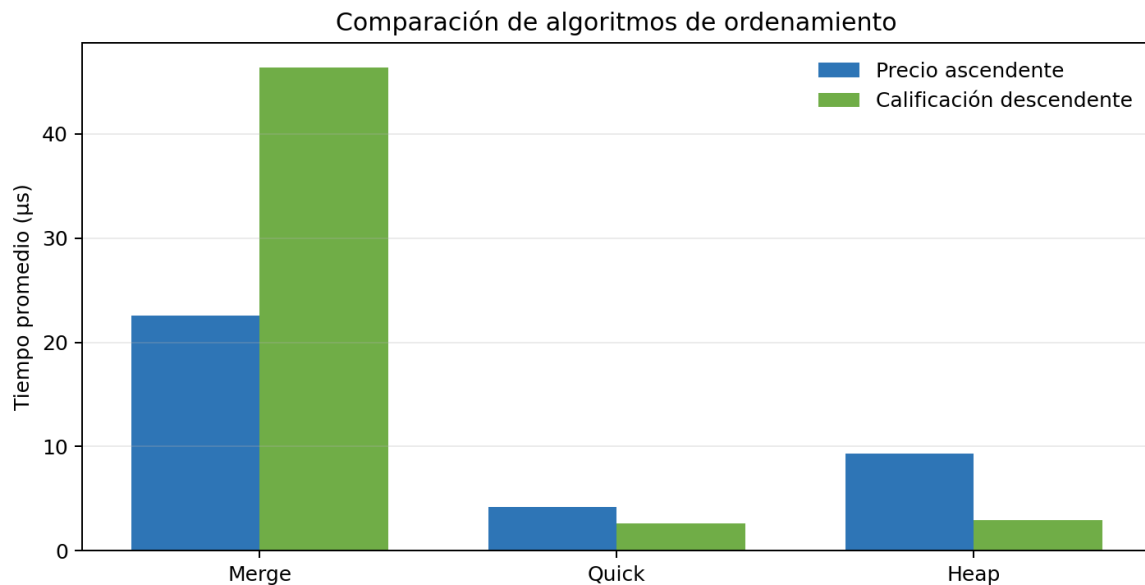
4. Resultados de ordenamiento

4.1 Precio ascendente

Algoritmo	Promedio (ns)	Promedio (us)
Merge Sort	22,540	22.540
Quick Sort	4,183	4.183
Heap Sort	9,354	9.354

4.2 Calificación descendente

Algoritmo	Promedio (ns)	Promedio (us)
Merge Sort	46,396	46.396
Quick Sort	2,660	2.660
Heap Sort	2,925	2.925



4.3 Analisis

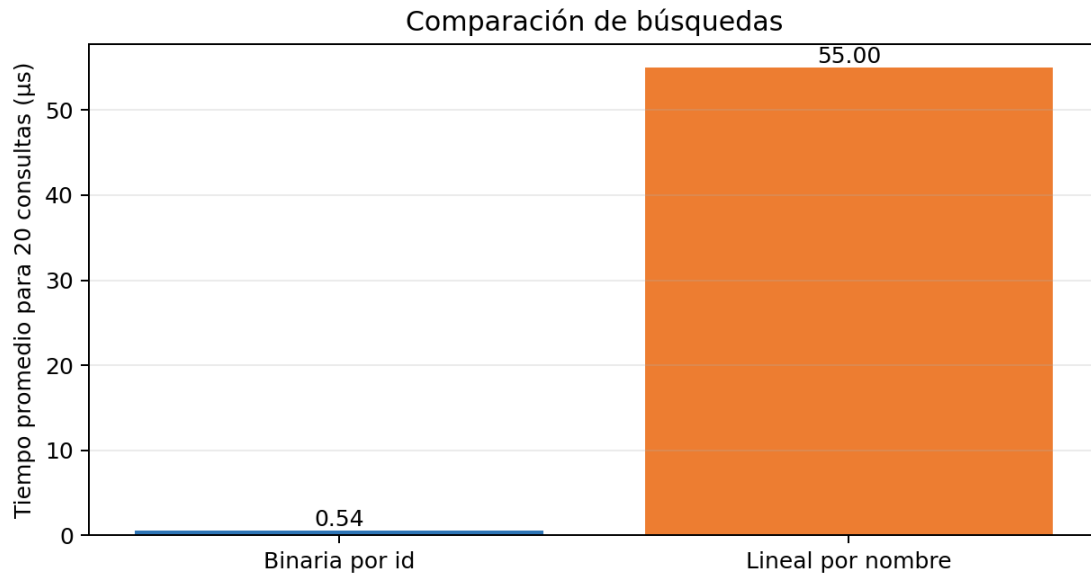
Por precio, Quick Sort obtuvo el menor promedio (3,531 ns), seguido por Heap Sort (5,536 ns) y Merge Sort (10,490 ns). Por calificación, Heap Sort fue el más rápido (2,998 ns), seguido por Quick Sort (7,438 ns) y Merge Sort (38,613 ns). No contradice la teoría: la notación asintótica no predice constantes, accesos a memoria, distribución de datos ni costo de recursión.

Objetos complejos frente a números simples. Existe una sobrecarga constante porque cada comparación invoca un método mediante Comparador. Sin embargo, las listas reubicar referencias en vez de copiar el objeto completo, por lo que la forma general de crecimiento no cambia. Con 50 elementos predominan las constantes y la JVM; para afirmar superioridad general se requieren tamaños mayores y varias distribuciones.

5. Búsqueda de productos

La lista se ordenó una vez por id mediante Merge Sort. Se hicieron diez consultas con ids existentes y diez con ids inexistentes. Para nombres se hicieron diez consultas de subcadenas con coincidencias y diez sin coincidencias. Cada lote de veinte se repitió 20,000 veces.

Método	20 consultas (ns)	20 consultas (us)
Binaria por id	542	0.542
Lineal por nombre	54,998	54.998



5.1 Por que la busqueda binaria es ideal para el id

El id es una clave unica, totalmente ordenable y comparable en tiempo constante. La busqueda binaria descarta la mitad del rango en cada paso y necesita $O(\log n)$ comparaciones; con 50 productos requiere como maximo unas seis iteraciones. Su condicion es mantener la lista ordenada por id. En un catalogo con muchas consultas y pocas modificaciones, el costo inicial de ordenar se amortiza rapidamente.

5.2 Subcadenas y alternativas de produccion

La busqueda lineal revisa los 50 nombres y evalua `contains()` en cada uno. Su costo simplificado es $O(n*m)$, donde m es el trabajo de comparar caracteres. Una subcadena no tiene la misma relacion de orden total que un id, por lo que no permite aplicar directamente busqueda binaria. Esto explica 52,523 ns frente a 455 ns.

En produccion pueden usarse indices de texto completo; tries para autocompletado; indices invertidos; normalizacion de mayusculas, tildes y sinonimos; o motores como Elasticsearch/OpenSearch. La eleccion depende de prefijos, subcadenas, tolerancia a errores y ranking.

6. Analisis y conclusiones

Elegir un algoritmo exige considerar complejidad teorica y forma real de los datos. Quick Sort fue el mas eficiente para precio en esta ejecucion, mientras Heap Sort lo fue para calificacion. La diferencia entre criterios se relaciona con distribucion de valores, calificaciones repetidas, pivote, cache y variabilidad; no significa que exista un ganador universal.

La busqueda binaria fue adecuada para ids por aprovechar una clave unica y ordenada. La busqueda de subcadenas es flexible, pero exige revisar todos los nombres y caracteres. En un sistema real, la estructura de almacenamiento y los patrones de consulta son tan importantes como el algoritmo aislado.

Trabajar con objetos obliga a separar algoritmo y criterio mediante Comparator, proteger el catalogo original y garantizar el orden previo por id. Tambien mostro la necesidad de calentar la JVM, repetir mediciones y no interpretar nanosegundos sin contexto. Como mejora futura se recomienda evaluar 1,000, 10,000 y 100,000 productos, datos casi ordenados y memoria.

7. Correspondencia con los requisitos

Requisito	Resultado
Clase Producto con seis atributos	Cumplido
50 objetos con ids unicos	Cumplido
Tres algoritmos	Merge, Quick y Heap
Precio ascendente y calificacion descendente	Cumplido
Tablas, analisis y teoria	Cumplido
10 ids existentes y 10 inexistentes	Cumplido
Busqueda binaria por id	Cumplido
Busqueda por nombre (opcional)	Incluida
10 subcadenas con y 10 sin resultados	Cumplido
Graficos (opcional)	Incluidos
Codigo documentado	Incluido

8. Estructura del codigo fuente

Archivo	Responsabilidad
Producto.java	Modelo inmutable
GeneradorProductos.java	Generacion reproducible
AlgoritmosOrdenamiento.java	Merge, Quick y Heap Sort
AlgoritmosBusqueda.java	Busqueda binaria y lineal
Principal.java	Pruebas, medicion, validacion y CSV
README.md	Instrucciones del proyecto

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2022). Introduction to Algorithms (4.a ed.). MIT Press.

Oracle. Java Platform, Standard Edition 17 API Specification: Comparator y System.nanoTime().

Anexo A. Código fuente documentado

Este anexo presenta el código completo utilizado en el proyecto. Los comentarios dentro de cada archivo explican con lenguaje sencillo la responsabilidad de las clases, las decisiones tomadas y el funcionamiento de los algoritmos. El mismo código se encuentra en la carpeta src del proyecto y puede ejecutarse desde Visual Studio Code.

Archivo	Que demuestra
Producto.java	Modelo de datos con los seis atributos solicitados.
GeneradorProductos.java	Creación de 50 productos coherentes e ids únicos.
AlgoritmosOrdenamiento.java	Implementación propia de Merge Sort, Quick Sort y Heap Sort.
AlgoritmosBusqueda.java	Busqueda binaria por id y lineal por subcadena.
Principal.java	Menu interactivo, comparadores, pruebas, medición, validación y archivos CSV.

A.1 Producto.java

Define el objeto principal del sistema. Sus atributos son privados y finales para evitar cambios accidentales durante las mediciones.

```
001 | package com.tienda.modelo;
002 |
003 | import java.util.Locale;
004 |
005 | /**
006 |  * Representa un producto del catálogo.
007 |  * Los algoritmos se dejaron en otras clases para mantener todo ordenado.
008 |  */
009 | public class Producto {
010 |     // id único usado en la búsqueda binaria
011 |     private final int id;
012 |     // nombre que se muestra en el catálogo
013 |     private final String nombre;
014 |     // double permite guardar precios con decimales
015 |     private final double precio;
016 |     private final String categoria;
017 |     private final int stock;
018 |     private final double calificacionPromedio;
019 |
020 |     public Producto(int id, String nombre, double precio, String categoria,
021 |         int stock, double calificacionPromedio) {
022 |         this.id = id;
023 |         this.nombre = nombre;
024 |         this.precio = precio;
025 |         this.categoria = categoria;
026 |         this.stock = stock;
027 |         this.calificacionPromedio = calificacionPromedio;
028 |     }
029 |
030 |     // métodos get para consultar los datos
031 |     public int getId() { return id; }
032 |     public String getNombre() { return nombre; }
033 |     public double getPrecio() { return precio; }
034 |     public String getCategoria() { return categoria; }
035 |     public int getStock() { return stock; }
036 |     public double getCalificacionPromedio() { return calificacionPromedio; }
037 |
038 |     @Override
039 |     public String toString() {
040 |         // formato usado para mostrar el producto en consola
041 |         return String.format(Locale.US,
042 |             "Producto{id=%d, nombre='%s', precio=%.2f, categoria='%s', stock=%d,
043 |             calificacion=%.1f}",
044 |             id, nombre, precio, categoria, stock, calificacionPromedio);
045 |     }
046 | }
```

A.2 GeneradorProductos.java

Construye el catalogo de prueba con una semilla fija. Los ids se crean sin repetirse y se mezclan antes de iniciar los algoritmos.

```
001 | package com.tienda.datos;
002 |
003 | import com.tienda.modelo.Producto;
004 | import java.util.ArrayList;
005 | import java.util.Collections;
006 | import java.util.List;
007 | import java.util.Random;
008 |
009 | /**
010 |  * Crea los productos utilizados en las pruebas.
011 |  * Se dejó separado para poder cambiar los datos fácilmente.
012 |  */
013 | public final class GeneradorProductos {
014 |     // cinco nombres por categoría para variar el catálogo
015 |     private static final String[] NOMBRES = {
016 |         "Auriculares Bluetooth", "Cámara Web HD", "Teclado Mecánico", "Ratón Inalámbrico",
017 |         "Monitor LED", "Camisa de Algodón", "Pantalón Deportivo", "Chaqueta Impermeable",
018 |         "Zapatillas Urbanas", "Vestido Casual", "Novela Histórica", "Manual de Programación",
019 |         "Libro de Cocina", "Atlas Universal", "Cuentos Infantiles", "Lámpara de Mesa",
020 |         "Juego de Sábanas", "Cafetera Eléctrica", "Organizador Modular", "Sartén Antiadherente"
021 |     };
022 |     private static final String[] MARCAS = {"Nova", "Orion", "Apex", "Lumen", "Terra"};
023 |     private static final String[] CATEGORIAS = {"Electrónica", "Ropa", "Libros", "Hogar"};
024 |
025 |     // no hace falta crear objetos de esta clase
026 |     private GeneradorProductos() { }
027 |
028 |     public static List<Producto> generar(int cantidad, long semilla) {
029 |         // la semilla hace que siempre se generen los mismos datos
030 |         Random random = new Random(semilla);
031 |         List<Integer> ids = new ArrayList<>();
032 |
033 |         // se crean ids únicos y luego se mezclan
034 |         for (int i = 1; i <= cantidad; i++) ids.add(1000 + i);
035 |         Collections.shuffle(ids, random);
036 |
037 |         List<Producto> productos = new ArrayList<>();
038 |         for (int i = 0; i < cantidad; i++) {
039 |             // combinamos nombre, marca y número para no repetir nombres
040 |             String base = NOMBRES[random.nextInt(NOMBRES.length)];
041 |             String nombre = base + " " + MARCAS[random.nextInt(MARCAS.length)] + " " + (i + 1);
042 |             String categoria = categoriaPara(base);
043 |
044 |             // valores aleatorios dentro de rangos razonables
045 |             double precio = Math.round((5.00 + random.nextDouble() * 495.00) * 100.0) / 100.0;
046 |             int stock = random.nextInt(201);
047 |             double calificacion = Math.round((1.0 + random.nextDouble() * 4.0) * 10.0) / 10.0;
048 |             productos.add(new Producto(ids.get(i), nombre, precio, categoria, stock, calificacion));
049 |         }
050 |         return productos;
051 |     }
052 |
053 |     private static String categoriaPara(String nombre) {
054 |         // cada grupo de cinco nombres pertenece a una categoría
055 |         int indice = java.util.Arrays.asList(NOMBRES).indexOf(nombre);
056 |         return CATEGORIAS[Math.min(indice / 5, CATEGORIAS.length - 1)];
057 |     }
058 | }
```


A.3 AlgoritmosOrdenamiento.java

Contiene las tres implementaciones solicitadas. Los metodos son genericos y usan Comparator para trabajar correctamente con objetos.

```
001 | package com.tienda.algoritmos;
002 |
003 | import java.util.Comparator;
004 | import java.util.List;
005 |
006 | /**
007 |  * Contiene los tres algoritmos de ordenamiento del proyecto.
008 |  * Comparator permite elegir el atributo que se quiere ordenar.
009 |  */
010 | public final class AlgoritmosOrdenamiento {
011 |     private AlgoritmosOrdenamiento() { }
012 |
013 |     public static <T> void mergeSort(List<T> lista, Comparator<T> comparador) {
014 |         // con menos de dos elementos no hay nada que ordenar
015 |         if (lista.size() < 2) return;
016 |         mergeSort(lista, comparador, 0, lista.size() - 1);
017 |     }
018 |
019 |     private static <T> void mergeSort(List<T> a, Comparator<T> c, int inicio, int fin) {
020 |         if (inicio >= fin) return;
021 |
022 |         // dividimos la lista en dos partes
023 |         int medio = (inicio + fin) >> 1;
024 |         mergeSort(a, c, inicio, medio);
025 |         mergeSort(a, c, medio + 1, fin);
026 |
027 |         // aquí juntamos las dos partes ordenadas
028 |         java.util.ArrayList<T> temp = new java.util.ArrayList<>(fin - inicio + 1);
029 |         int i = inicio, j = medio + 1;
030 |         while (i <= medio && j <= fin)
031 |             temp.add(c.compare(a.get(i), a.get(j)) <= 0 ? a.get(i++) : a.get(j++));
032 |         while (i <= medio) temp.add(a.get(i++));
033 |         while (j <= fin) temp.add(a.get(j++));
034 |         // copiamos el resultado a la lista original
035 |         for (int k = 0; k < temp.size(); k++) a.set(inicio + k, temp.get(k));
036 |     }
037 |
038 |     public static <T> void quickSort(List<T> lista, Comparator<T> comparador) {
039 |         quickSort(lista, comparador, 0, lista.size() - 1);
040 |     }
041 |
042 |     private static <T> void quickSort(List<T> a, Comparator<T> c, int bajo, int alto) {
043 |         if (bajo >= alto) return;
044 |
045 |         // separamos los elementos usando el pivote
046 |         int p = particionar(a, c, bajo, alto);
047 |         quickSort(a, c, bajo, p - 1);
048 |         quickSort(a, c, p + 1, alto);
049 |     }
050 |
051 |     private static <T> int particionar(List<T> a, Comparator<T> c, int bajo, int alto) {
052 |         // usamos el último elemento como pivote
053 |         T pivote = a.get(alto);
054 |         int i = bajo - 1;
055 |         for (int j = bajo; j < alto; j++) {
056 |             // los menores se mueven hacia la izquierda
057 |             if (c.compare(a.get(j), pivote) <= 0) intercambiar(a, ++i, j);
058 |         }
059 |         intercambiar(a, i + 1, alto);
060 |         return i + 1;
061 |     }
062 |
063 |     public static <T> void heapSort(List<T> lista, Comparator<T> comparador) {
064 |         int n = lista.size();
065 |
066 |         // primero construimos el montículo
067 |         for (int i = n / 2 - 1; i >= 0; i--) ajustarMonticulo(lista, n, i, comparador);
068 |
069 |         // movemos el mayor al final y ajustamos otra vez
070 |         for (int fin = n - 1; fin > 0; fin--) {
071 |             intercambiar(lista, 0, fin);
072 |             ajustarMonticulo(lista, fin, 0, comparador);
073 |         }
074 |     }
075 |
076 |     private static <T> void ajustarMonticulo(List<T> a, int n, int raiz, Comparator<T> c) {
077 |         // posiciones de los hijos de la raíz
078 |         int mayor = raiz, izquierdo = 2 * raiz + 1, derecho = 2 * raiz + 2;
079 |         if (izquierdo < n && c.compare(a.get(izquierdo), a.get(mayor)) > 0) mayor = izquierdo;
080 |         if (derecho < n && c.compare(a.get(derecho), a.get(mayor)) > 0) mayor = derecho;
081 |         if (mayor != raiz) {
082 |             // intercambiamos y seguimos revisando
083 |             intercambiar(a, raiz, mayor);
084 |             ajustarMonticulo(a, n, mayor, c);
085 |         }
086 |     }
087 |
088 |     private static <T> void intercambiar(List<T> a, int i, int j) {
089 |         // intercambio sencillo entre dos posiciones
090 |         T temporal = a.get(i); a.set(i, a.get(j)); a.set(j, temporal);
091 |     }
092 | }
```

A.4 AlgoritmosBusqueda.java

Incluye la búsqueda binaria, que requiere ids ordenados, y la búsqueda lineal por coincidencia parcial de nombre.

```
001 | package com.tienda.algoritmos;
002 |
003 | import com.tienda.modelo.Producto;
004 | import java.util.ArrayList;
005 | import java.util.List;
006 | import java.util.Locale;
007 |
008 | public final class AlgoritmosBusqueda {
009 |     private AlgoritmosBusqueda() { }
010 |
011 |     /**
012 |      * Busca un producto por id.
013 |      * La lista debe estar ordenada antes de usar este método.
014 |      */
015 |     public static Producto busquedaBinariaPorId(List<Producto> productos, int id) {
016 |         int izquierda = 0, derecha = productos.size() - 1;
017 |         while (izquierda <= derecha) {
018 |             // calculamos la posición central
019 |             int medio = (izquierda + derecha) >> 1;
020 |             Producto actual = productos.get(medio);
021 |             if (actual.getId() == id) return actual;
022 |
023 |             // descartamos la mitad donde no puede estar el id
024 |             if (actual.getId() < id) izquierda = medio + 1;
025 |             else derecha = medio - 1;
026 |         }
027 |         // si no aparece se devuelve null
028 |         return null;
029 |     }
030 |
031 |     public static List<Producto> busquedaLinealPorNombre(List<Producto> productos, String subcadena) {
032 |         // pasamos todo a minúsculas para comparar mejor
033 |         String consulta = subcadena.toLowerCase(Locale.ROOT);
034 |         List<Producto> resultados = new ArrayList<>();
035 |
036 |         // revisamos todos los productos uno por uno
037 |         for (Producto producto : productos)
038 |             if (producto.getNombre().toLowerCase(Locale.ROOT).contains(consulta))
039 |                 resultados.add(producto);
040 |         return resultados;
041 |     }
042 | }
```

A.5 Principal.java

Incluye el menu interactivo y coordina el experimento completo: muestra y ordena productos, realiza busquedas, mide tiempos, valida resultados y exporta los CSV.

```
001 | package com.tienda.app;
002 |
003 | import com.tienda.algoritmos.AlgoritmosBusqueda;
004 | import com.tienda.algoritmos.AlgoritmosOrdenamiento;
005 | import com.tienda.datos.GeneradorProductos;
006 | import com.tienda.modelo.Producto;
007 | import java.io.IOException;
008 | import java.nio.file.Files;
009 | import java.nio.file.Path;
010 | import java.nio.file.Paths;
011 | import java.util.ArrayList;
012 | import java.util.Comparator;
013 | import java.util.List;
014 | import java.util.Locale;
015 | import java.util.Random;
016 | import java.util.Scanner;
017 | import java.util.function.BiConsumer;
018 |
019 | public class Principal {
020 |     // cantidad de veces que se repite cada prueba
021 |     private static final int REPETICIONES = 5_000;
022 |
023 |     // comparadores usados para cada tipo de orden
024 |     private static final Comparator<Producto> POR_PRECIO =
025 |         Comparator.comparingDouble(Producto::getPrecio);
026 |     private static final Comparator<Producto> POR_CALIFICACION =
027 |         Comparator.comparingDouble(Producto::getCalificacionPromedio).reversed();
028 |     private static final Comparator<Producto> POR_ID = Comparator.comparingInt(Producto::getId);
029 |
030 |     public static void main(String[] args) throws IOException {
031 |         // usamos punto para los números decimales
032 |         Locale.setDefault(Locale.US);
033 |
034 |         // generamos los 50 productos del proyecto
035 |         List<Producto> productos = GeneradorProductos.generar(50, 20260815L);
036 |         verificarDatos(productos);
037 |
038 |         // guardamos el catálogo para poder abrirlo en Excel
039 |         Files.createDirectories(Paths.get("resultados"));
040 |         Files.writeString(Paths.get("resultados", "productos.csv"), productosCsv(productos));
041 |
042 |         // abrimos el menú principal
043 |         mostrarMenu(productos);
044 |     }
045 |
046 |     /** Muestra el menú hasta que el usuario decida salir. */
047 |     private static void mostrarMenu(List<Producto> productos) throws IOException {
048 |         Scanner teclado = new Scanner(System.in);
049 |         int opcion = 0;
050 |
051 |         while (opcion != 7) {
052 |             System.out.println("\n=====");
053 |             System.out.println(" SISTEMA DE PRODUCTOS - ALGORITMOS Y BÚSQUEDAS");
054 |             System.out.println("=====");
055 |             System.out.println("1. Mostrar los 50 productos");
056 |             System.out.println("2. Ordenar productos por precio (ascendente)");
057 |             System.out.println("3. Ordenar productos por calificación (descendente)");
058 |             System.out.println("4. Buscar un producto por ID");
059 |             System.out.println("5. Buscar productos por nombre");
060 |             System.out.println("6. Ejecutar comparación de algoritmos");
061 |             System.out.println("7. Salir");
062 |             System.out.print("Seleccione una opción: ");
063 |
064 |             try {
065 |                 opcion = Integer.parseInt(teclado.nextLine().trim());
066 |             } catch (NumberFormatException e) {
067 |                 opcion = 0;
068 |             }
069 |
070 |             switch (opcion) {
071 |                 case 1:
072 |                     mostrarProductos(productos);
073 |                     break;
074 |                 case 2:
075 |                     mostrarOrdenados(productos, POR_PRECIO, "PRODUCTOS ORDENADOS POR PRECIO");
076 |                     break;
077 |                 case 3:
078 |                     mostrarOrdenados(productos, POR_CALIFICACION,
079 |                                     "PRODUCTOS ORDENADOS POR CALIFICACIÓN");
080 |                     break;
081 |                 case 4:
082 |                     buscarIdDesdeMenu(productos, teclado);
083 |                     break;
084 |                 case 5:
085 |                     buscarNombreDesdeMenu(productos, teclado);
086 |                     break;
087 |                 case 6:
088 |                     ejecutarComparacionCompleta(productos);
089 |                     break;
090 |                 case 7:
091 |                     System.out.println("Programa finalizado. ¡Gracias!");
092 |                     break;
093 |                 default:
094 |                     System.out.println("Opción inválida. Escriba un número del 1 al 7.");
095 |             }
096 |         }
097 |     }
098 | }
```

```

095 |         teclado.close();
096 |     }
097 |
098 |     /** Muestra los productos en forma de tabla. */
099 |     private static void mostrarProductos(List<Producto> productos) {
100 |         System.out.println("\nID      NOMBRE      PRECIO      CATEGORÍA
101 |         STOCK  CALIF."");
102 |         System.out.println("-----");
103 |         for (Producto p : productos) {
104 |             System.out.printf("%-5d %-39s B/.-8.2f %-15s %-6d %.1f%n",
105 |                 p.getId(), p.getNombre(), p.getPrecio(), p.getCategoria(),
106 |                 p.getStock(), p.getCalificacionPromedio());
107 |         }
108 |
109 |         /** Ordena una copia para no cambiar la lista original. */
110 |         private static void mostrarOrdenados(List<Producto> productos,
111 |             Comparator<Producto> comparador, String titulo) {
112 |             List<Producto> copia = new ArrayList<>(productos);
113 |             AlgoritmosOrdenamiento.mergeSort(copia, comparador);
114 |             System.out.println("\n=== " + titulo + " ===");
115 |             mostrarProductos(copia);
116 |         }
117 |
118 |         private static void buscarIdDesdeMenu(List<Producto> productos, Scanner teclado) {
119 |             System.out.print("Escriba el ID que desea buscar (1001-1050): ");
120 |             try {
121 |                 int id = Integer.parseInt(teclado.nextLine().trim());
122 |                 List<Producto> porId = new ArrayList<>(productos);
123 |                 AlgoritmosOrdenamiento.mergeSort(porId, POR_ID);
124 |                 Producto encontrado = AlgoritmosBusqueda.busquedaBinariaPorId(porId, id);
125 |                 if (encontrado == null) System.out.println("No existe un producto con ese ID.");
126 |                 else {
127 |                     System.out.println("Producto encontrado:");
128 |                     System.out.println(encontrado);
129 |                 }
130 |             } catch (NumberFormatException e) {
131 |                 System.out.println("El ID debe ser un número entero.");
132 |             }
133 |         }
134 |
135 |         private static void buscarNombreDesdeMenu(List<Producto> productos, Scanner teclado) {
136 |             System.out.print("Escriba el nombre o una parte del nombre: ");
137 |             String texto = teclado.nextLine().trim();
138 |             List<Producto> encontrados = AlgoritmosBusqueda.busquedaLinealPorNombre(productos, texto);
139 |             if (encontrados.isEmpty()) System.out.println("No se encontraron coincidencias.");
140 |             else {
141 |                 System.out.println("Se encontraron " + encontrados.size() + " producto(s):");
142 |                 mostrarProductos(encontrados);
143 |             }
144 |         }
145 |
146 |         /** Ejecuta las mediciones y actualiza tiempos.csv. */
147 |         private static void ejecutarComparacionCompleta(List<Producto> productos) throws IOException {
148 |             StringBuilder csv = new StringBuilder("tipo,criterio,algoritmo,tiempo_promedio_ns\n");
149 |             System.out.println("\n=== ORDENAMIENTO: promedio de " + REPETICIONES + " ejecuciones (ns)
150 |             ===");
151 |             medirCriterio(productos, "Precio ascendente", POR_PRECIO, csv);
152 |             medirCriterio(productos, "Calificación descendente", POR_CALIFICACION, csv);
153 |
154 |             List<Producto> porId = new ArrayList<>(productos);
155 |             AlgoritmosOrdenamiento.mergeSort(porId, POR_ID);
156 |             ejecutarBusquedas(porId, csv);
157 |             Files.writeString(Paths.get("resultados", "tiempos.csv"), csv.toString());
158 |             System.out.println("\nMediciones guardadas en resultados/tiempos.csv");
159 |         }
160 |
161 |         private static void medirCriterio(List<Producto> original, String criterio,
162 |             Comparator<Producto> comparador, StringBuilder csv) {
163 |             // probamos los tres algoritmos con el mismo criterio
164 |             medir("Merge Sort", original, comparador, AlgoritmosOrdenamiento::mergeSort, criterio, csv);
165 |             medir("Quick Sort", original, comparador, AlgoritmosOrdenamiento::quickSort, criterio, csv);
166 |             medir("Heap Sort", original, comparador, AlgoritmosOrdenamiento::heapSort, criterio, csv);
167 |         }
168 |
169 |         private static void medir(String nombre, List<Producto> original, Comparator<Producto> comparador,
170 |             BiConsumer<List<Producto>, Comparator<Producto>> algoritmo, String criterio, StringBuilder
171 |             csv) {
172 |             // hacemos unas ejecuciones antes de comenzar a medir
173 |             for (int i = 0; i < 500; i++) algoritmo.accept(new ArrayList<>(original), comparador);
174 |             long total = 0;
175 |             List<Producto> ultima = null;
176 |             for (int i = 0; i < REPETICIONES; i++) {
177 |                 // cada prueba recibe una copia de la lista original
178 |                 ultima = new ArrayList<>(original);
179 |
180 |                 // el cronómetro solo mide el ordenamiento
181 |                 long inicio = System.nanoTime();
182 |                 algoritmo.accept(ultima, comparador);
183 |                 total += System.nanoTime() - inicio;
184 |             }
185 |             // comprobamos que el resultado esté bien ordenado
186 |             if (!estaOrdenada(ultima, comparador)) throw new IllegalStateException(nombre + " produjo un
187 |             orden incorrecto");
188 |             long promedio = total / REPETICIONES;
189 |             System.out.printf("%-25s %-12s %d ns%n", criterio, nombre, promedio);
190 |             csv.append("ordenamiento, ").append(criterio).append(',').append(nombre).append(',').append(prom
191 |             edio).append('\n');
192 |         }
193 |
194 |         private static void ejecutarBusquedas(List<Producto> porId, StringBuilder csv) {
195 |             Random random = new Random(77);
196 |             int[] existentes = new int[10], inexistentes = new int[10];
197 |
198 |             // preparamos 10 ids existentes y 10 inexistentes

```

```

195 |         for (int i = 0; i < 10; i++) { existentes[i] = porId.get(random.nextInt(50)).getId();
196 |             inexistentes[i] = 2001 + i; }
197 |         long binaria = medirBinaria(porId, existentes, inexistentes, 20_000);
198 |         // consultas con resultados y sin resultados
199 |         String[] conResultado = {"camisa", "libro", "lámpara", "nova", "orion", "cámara", "hogar",
200 |             "algodón", "mesa", "urbana"};
201 |         String[] sinResultado = {"teléfono", "bicicleta", "reloj", "jardín", "perfume", "consola",
202 |             "zapato rojo", "xyz", "tablet", "nevera"};
203 |         long lineal = medirLineal(porId, conResultado, sinResultado, 20_000);
204 |         System.out.println("\n=== BÚSQUEDAS: tiempo promedio total para 20 consultas ===");
205 |         System.out.printf("Binaria por id: %d ns\n", binaria);
206 |         System.out.printf("Lineal por subcadena: %d ns\n", lineal);
207 |         csv.append("busqueda,20 consultas,Binaria por id,").append(binaria).append('\n');
208 |         csv.append("busqueda,20 consultas,Lineal por nombre,").append(lineal).append('\n');
209 |     }
210 |
211 |     private static long medirBinaria(List<Producto> lista, int[] si, int[] no, int repeticiones) {
212 |         // contamos los ids encontrados para validar la prueba
213 |         long total = 0; int encontrados = 0;
214 |         for (int r = 0; r < repeticiones; r++) {
215 |             long inicio = System.nanoTime();
216 |             for (int id : si) if (AlgoritmosBusqueda.busquedaBinariaPorId(lista, id) != null)
217 |                 encontrados++;
218 |             for (int id : no) if (AlgoritmosBusqueda.busquedaBinariaPorId(lista, id) != null)
219 |                 encontrados++;
220 |             total += System.nanoTime() - inicio;
221 |         }
222 |         if (encontrados != repeticiones * 10) throw new IllegalStateException("Error en búsqueda
223 |             binaria");
224 |         return total / repeticiones;
225 |     }
226 |
227 |     private static long medirLineal(List<Producto> lista, String[] si, String[] no, int repeticiones) {
228 |         // sumamos las coincidencias encontradas
229 |         long total = 0; int coincidencias = 0;
230 |         for (int r = 0; r < repeticiones; r++) {
231 |             long inicio = System.nanoTime();
232 |             for (String q : si) coincidencias += AlgoritmosBusqueda.busquedaLinealPorNombre(lista,
233 |                 q).size();
234 |             for (String q : no) coincidencias += AlgoritmosBusqueda.busquedaLinealPorNombre(lista,
235 |                 q).size();
236 |             total += System.nanoTime() - inicio;
237 |         }
238 |         if (coincidencias == 0) throw new IllegalStateException("Las consultas válidas no produjeron
239 |             coincidencias");
240 |         return total / repeticiones;
241 |     }
242 |
243 |     private static boolean estaOrdenada(List<Producto> a, Comparator<Producto> c) {
244 |         // revisamos cada pareja de elementos
245 |         for (int i = 1; i < a.size(); i++) if (c.compare(a.get(i - 1), a.get(i)) > 0) return false;
246 |         return true;
247 |     }
248 |
249 |     private static void verificarDatos(List<Producto> p) {
250 |         // comprobamos que sean 50 productos con ids únicos
251 |         if (p.size() != 50 || p.stream().map(Producto::getId).distinct().count() != 50)
252 |             throw new IllegalStateException("Deben existir 50 productos con id único");
253 |     }
254 |
255 |     private static String productosCsv(List<Producto> productos) {
256 |         // convertimos los productos al formato CSV
257 |         StringBuilder s = new StringBuilder("id,nombre,precio,categoria,stock,calificacionPromedio\n");
258 |         for (Producto p : productos) s.append(String.format(Locale.US, "%d,\"%s\",%.2f,%s,%d,%.1f\n",
259 |             p.getId(), p.getNombre(), p.getPrecio(), p.getCategoria(), p.getStock(),
260 |             p.getCalificacionPromedio()));
261 |         return s.toString();
262 |     }
263 | }

```